

Module 5

Lab - Working with 1D Numpy Arrays

Code:

```
import numpy as np
```

Output:

No output

Code:

```
a = np.array([0, 1, 2, 3, 4])
```

```
a
```

Output:

```
array([0, 1, 2, 3, 4])
```

Code:

```
print("a[0]:", a[0])
```

```
print("a[1]:", a[1])
```

```
print("a[2]:", a[2])
```

```
print("a[3]:", a[3])
```

```
print("a[4]:", a[4])
```

Output:

```
a[0]: 0
```

```
a[1]: 1
```

```
a[2]: 2
```

```
a[3]: 3
```

```
a[4]: 4
```

Code:

```
print(np.__version__)
```

Output:

```
1.26.4
```

Code:

```
type(a)
```

Output:

```
numpy.ndarray
```

Code:

```
a.dtype
```

Output:

```
dtype('int32')
```

Code:

```
b = np.array([3.1, 11.02, 6.2, 213.2, 5.2])
```

```
type(b)
```

```
b.dtype
```

Output:

```
dtype('float64')
```

Code:

```
c = np.array([20, 1, 2, 3, 4])
```

```
c
```

Output:

```
array([20, 1, 2, 3, 4])
```

Code:

```
c[0] = 100
```

```
c
```

Output:

```
array([100, 1, 2, 3, 4])
```

Code:

```
c[4] = 0
```

```
c
```

Output:

```
array([100, 1, 2, 3, 0])
```

Code:

```
a = np.array([10, 2, 30, 40, 50])
```

```
a[1]=20
```

```
a
```

Output:

```
array([10, 20, 30, 40, 50])
```

Code:

```
d = c[1:4]
```

```
d
```

Output:

```
array([1, 2, 3])
```

Code:

```
c[3:5] = 300, 400
```

```
c
```

Output:

```
array([100, 1, 2, 300, 400])
```

Code:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
print(arr[1:5:2])
```

Output:

```
[2 4]
```

Code:

```
print(arr[:4])
```

Output:

```
[1 2 3 4]
```

Code:

```
print(arr[4:])
```

Output:

```
[5 6 7]
```

Code:

```
print(arr[1:5:])
```

Output:

```
[2 3 4 5]
```

Code:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
print(arr[1:8:2])
```

Output:

```
[2 4 6 8]
```

Code:

```
select = [0, 2, 3, 4]
```

```
select
```

Output:

```
[0, 2, 3, 4]
```

Code:

```
d = c[select]
```

```
d
```

Output:

```
array([100,  2, 300, 400])
```

Code:

```
c[select] = 100000
```

```
c
```

Output:

```
array([100000,  1, 100000, 100000, 100000])
```

Code:

```
a = np.array([0, 1, 2, 3, 4])
```

```
a
```

Output:

```
array([0, 1, 2, 3, 4])
```

Code:

```
a.size
```

Output:

```
5
```

Code:

```
a.ndim
```

Output:

```
1
```

Code:

```
a.shape
```

Output:

```
(5,)
```

Code:

```
b = np.array([10, 20, 30, 40, 50, 60, 70])
```

```
b.size
```

```
b.ndim
```

```
b.shape
```

Output:

```
(7,)
```

Code:

```
a = np.array([1, -1, 1, -1])
```

Output:

```
No output
```

Code:

```
mean = a.mean()
```

```
mean
```

Output:

```
0.0
```

Code:

```
standard_deviation=a.std()
```

```
standard_deviation
```

Output:

```
1.0
```

Code:

```
b = np.array([-1, 2, 3, 4, 5])
```

```
b
```

Output:

```
array([-1, 2, 3, 4, 5])
```

Code:

```
max_b = b.max()
```

```
max_b
```

Output:

```
5
```

Code:

```
min_b = b.min()
```

```
min_b
```

Output:

```
-1
```

Code:

```
c = np.array([-10, 201, 43, 94, 502])
```

```
max_c = c.max()
```

```
max_c
```

```
min_c = c.min()
```

```
min_c
```

```
Sum = (max_c + min_c)
```

```
Sum
```

Output:

```
492
```

Code:

```
u = np.array([1, 0])
```

```
u
```

Output:

```
array([1, 0])
```

Code:

```
v = np.array([0, 1])
```

```
v
```

Output:

```
array([0, 1])
```

Code:

```
z = np.add(u, v)
```

```
z
```

Output:

```
array([1, 1])
```

Code:

```
import time
```

```
import sys
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def Plotvec1(u, z, v):
```

```

ax = plt.axes()
ax.arrow(0, 0, *u, head_width=0.05, color='r', head_length=0.1)
plt.text(*(u + 0.1), 'u')#Adds the text u to the Axes

ax.arrow(0, 0, *v, head_width=0.05, color='b', head_length=0.1)
plt.text(*(v + 0.1), 'v')#Adds the text v to the Axes

ax.arrow(0, 0, *z, head_width=0.05, head_length=0.1)
plt.text(*(z + 0.1), 'z')
plt.ylim(-2, 2)
plt.xlim(-2, 2)

```

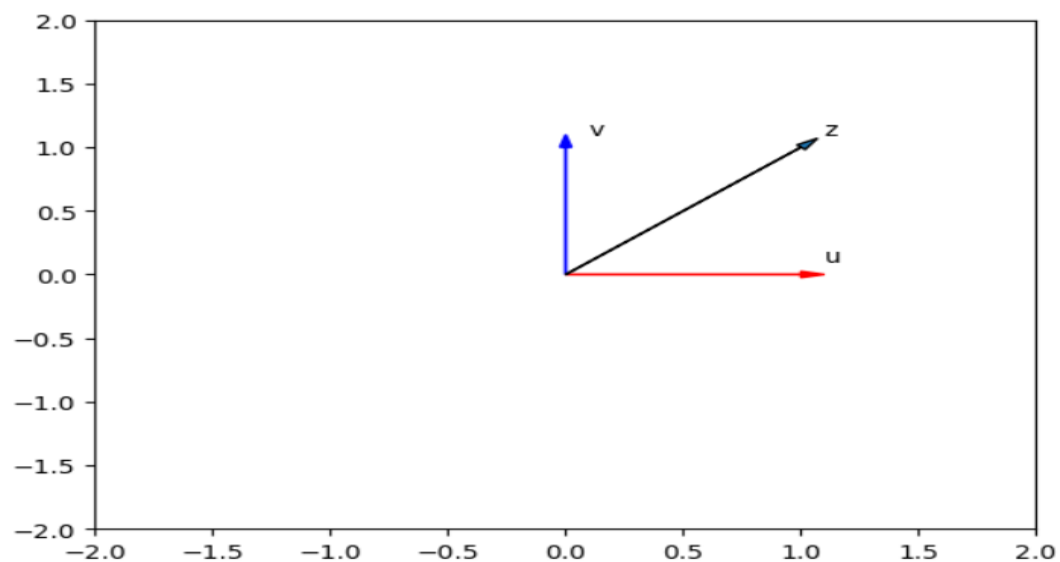
Output:

No output

Code:

Plotvec1(u, z, v)

Output:



Code:

```
arr1 = np.array([10, 11, 12, 13, 14, 15])  
arr2 = np.array([20, 21, 22, 23, 24, 25])  
arr3 = np.add(arr1, arr2)  
  
arr3
```

Output:

```
array([30, 32, 34, 36, 38, 40])
```

Code:

```
a = np.array([10, 20, 30])  
  
a
```

Output:

```
array([10, 20, 30])
```

Code:

```
b = np.array([5, 10, 15])  
  
b
```

Output:

```
array([ 5, 10, 15])
```

Code:

```
c = np.subtract(a, b)  
  
print(c)
```

Output:

```
[ 5 10 15]
```

Code:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])  
arr2 = np.array([20, 21, 22, 23, 24, 25])  
arr3 = np.subtract(arr1, arr2)  
  
arr3
```

Output:

```
array([-10, -1,  8, 17, 26, 35])
```

Code:

```
x = np.array([1, 2])
```

```
x
```

Output:

```
array([1, 2])
```

Code:

```
y = np.array([2, 1])
```

```
y
```

Output:

```
array([2, 1])
```

Code:

```
z = np.multiply(x, y)
```

```
z
```

Output:

```
array([2, 2])
```

Code:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])
```

```
arr2 = np.array([2, 1, 2, 3, 4, 5])
```

```
arr3 = np.multiply(arr1, arr2)
```

```
arr3
```

Output:

```
array([ 20,  20,  60, 120, 200, 300])
```

Code:

```
a = np.array([10, 20, 30])
```

```
a
```

Output:

```
array([10, 20, 30])
```

Code:

```
b = np.array([2, 10, 5])
```

```
b
```

Output:

```
array([ 2, 10,  5])
```

Code:

```
c = np.divide(a, b)
```

```
c
```

Output:

```
array([5., 2., 6.])
```

Code:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])
```

```
arr2 = np.array([3, 5, 10, 8, 2, 33])
```

```
arr3 = np.divide(arr1, arr2)
```

```
arr3
```

Output:

```
array([ 3.33333333,  4.        ,  3.        ,  5.        , 25.        ,  
       1.81818182])
```

Code:

```
X = np.array([1, 2])
```

```
Y = np.array([3, 2])
```

Output:

```
No output
```

Code:

```
np.dot(X, Y)
```

Output:

7

Code:

```
print(X[0])
```

```
print(X[1])
```

Output:

1

2

Code:

```
print(Y[0])
```

```
print(Y[1])
```

Output:

3

2

Code:

```
arr1 = np.array([3, 5])
```

```
arr2 = np.array([2, 4])
```

```
arr3 = np.dot(arr1, arr2)
```

```
arr3
```

Output:

26

Code:

```
u = np.array([1, 2, 3, -1])
```

```
u
```

Output:

```
array([ 1,  2,  3, -1])
```

Code:

```
u + 1
```

Output:

```
array([2, 3, 4, 0])
```

Code:

```
arr = np.array([1, 2, 3, -1])
```

```
arr + 5
```

Output:

```
array([6, 7, 8, 4])
```

Code:

```
np.pi
```

Output:

```
3.141592653589793
```

Code:

```
x = np.array([0, np.pi/2, np.pi])
```

Output:

```
No output
```

Code:

```
y = np.sin(x)
```

```
y
```

Output:

```
array([0.0000000e+00, 1.0000000e+00, 1.2246468e-16])
```

Code:

```
np.linspace(-2, 2, num=5)
```

Output:

```
array([-2., -1., 0., 1., 2.])
```

Code:

```
np.linspace(-2, 2, num=9)
```

Output:

```
array([-2. , -1.5, -1. , -0.5, 0. , 0.5, 1. , 1.5, 2. ])
```

Code:

```
x = np.linspace(0, 2*np.pi, num=100)
```

Output:

No output

Code:

```
y = np.sin(x)
```

Output:

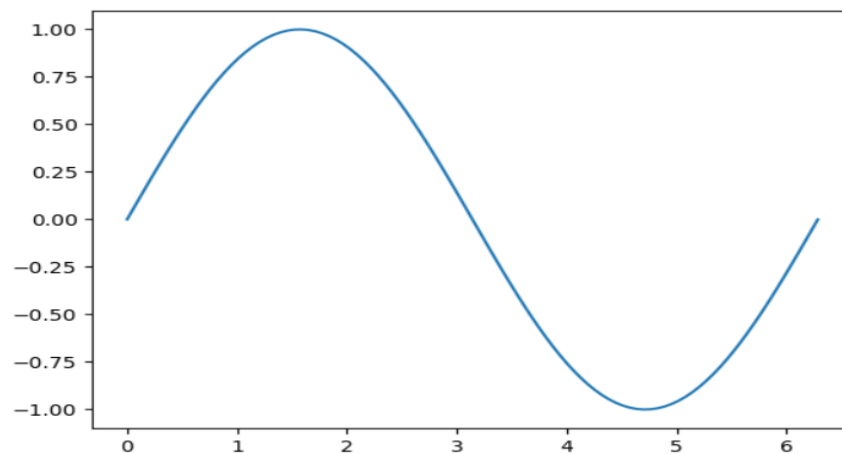
No output

Code:

```
plt.plot(x, y)
```

Output:

[<matplotlib.lines.Line2D at 0x44ff998>]

**Code:**

```
np.linspace(5, 4, num=6)
```

Output:

```
array([5. , 4.8, 4.6, 4.4, 4.2, 4. ])
```

Code:

```
arr1 = np.array([1, 2, 3])
```

```
print(arr1)
```

Output:

```
[1 2 3]
```

Code:

```
for x in arr1:
```

```
    print(x)
```

Output:

```
1
```

```
2
```

```
3
```

Quiz on 1D Numpy Array**Code:**

```
u = np.array([1, 0])
```

```
v = np.array([0, 1])
```

```
u - v
```

Output:

```
array([ 1, -1])
```

Code:

```
z = np.array([2, 4])
```

```
-2 * z
```

Output:

```
array([-4, -8])
```

Code:

```
a = np.array([1, 2, 3, 4, 5])
```

```
b = np.array([1, 0, 1, 0, 1])
```

```
a * b
```

Output:

```
array([1, 0, 3, 0, 5])
```

Code:

```
import time
```

```
import sys
```

```

import numpy as np

import matplotlib.pyplot as plt

def Plotvec2(a,b):

    ax = plt.axes()

    ax.arrow(0, 0, *a, head_width=0.05, color='r', head_length=0.1)

    plt.text(*(a + 0.1), 'a')

    ax.arrow(0, 0, *b, head_width=0.05, color='b', head_length=0.1)

    plt.text(*(b + 0.1), 'b')

    plt.ylim(-2, 2)#set the ylim to bottom(-2), top(2)

    plt.xlim(-2, 2)#set the xlim to left(-2), right(2)

```

Output:

No output

Code:

```

a = np.array([-1, 1])

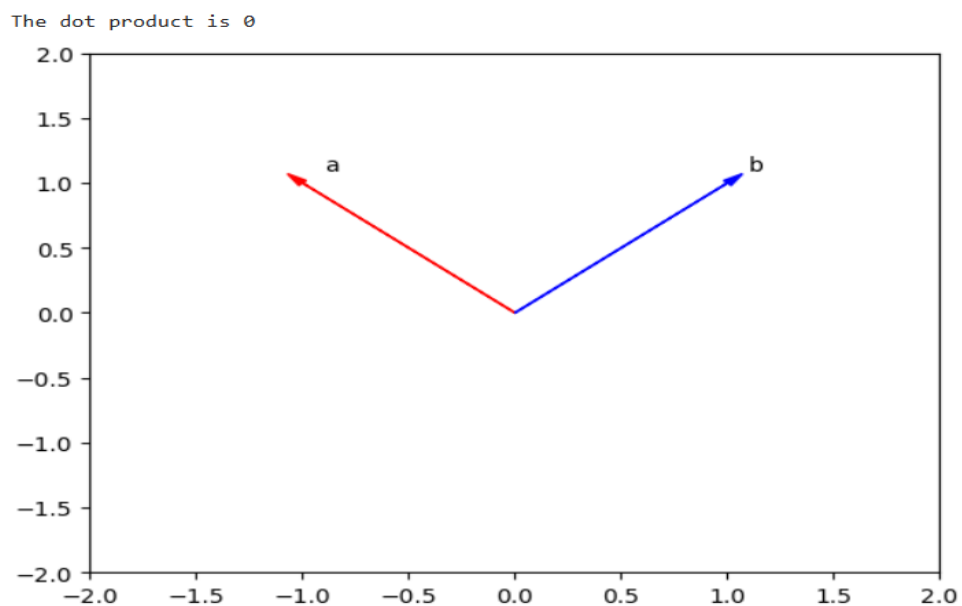
b = np.array([1, 1])

Plotvec2(a, b)

print("The dot product is", np.dot(a,b))

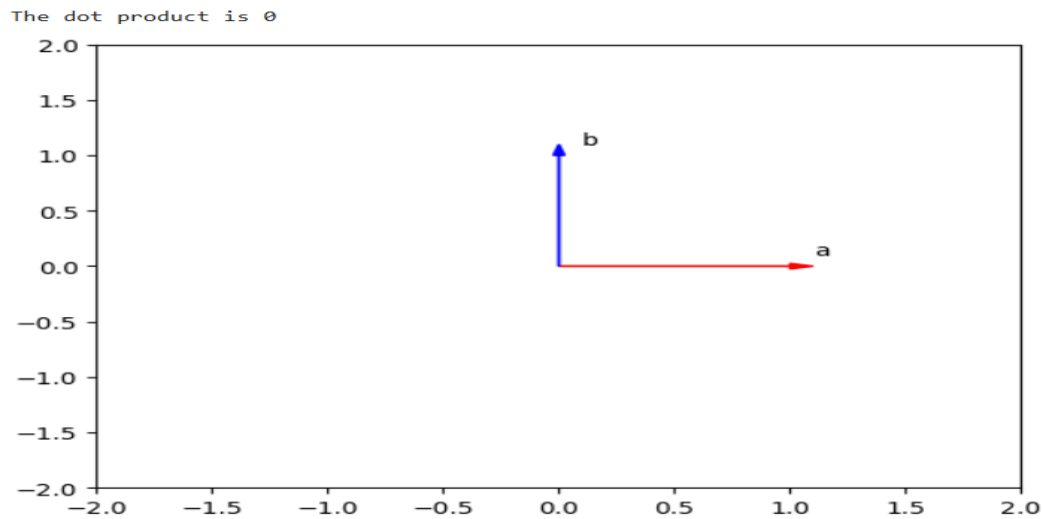
```

Output:

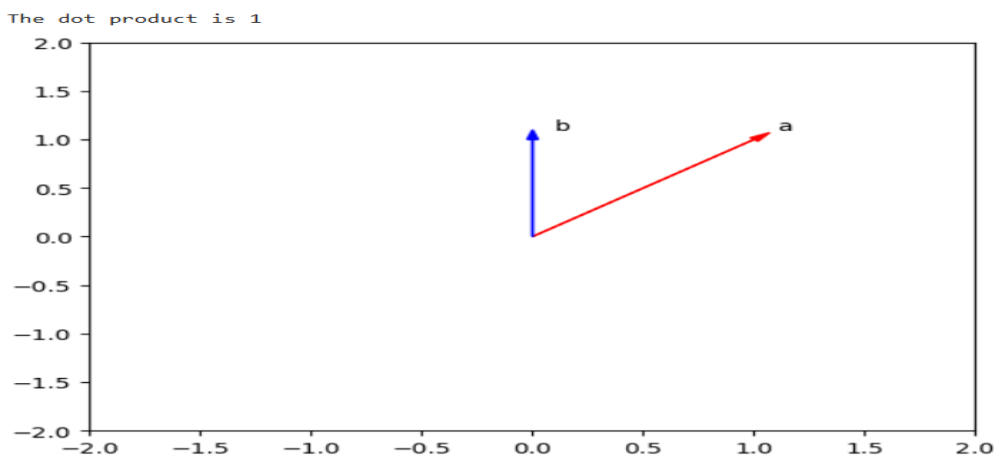


Code:

```
a = np.array([1, 0])  
b = np.array([0, 1])  
Plotvec2(a, b)  
print("The dot product is", np.dot(a, b))
```

Output:**Code:**

```
a = np.array([1, 1])  
b = np.array([0, 1])  
Plotvec2(a, b)  
print("The dot product is", np.dot(a, b))
```

Output:

Code:

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([8, 9, 10])
arr3 = np.add(arr1, arr2)
arr3
arr4 = np.subtract(arr1, arr2)
arr4
arr5 = np.multiply(arr1, arr2)
arr5
arr6 = np.divide(arr1, arr2)
arr6
arr7 = np.dot(arr1, arr2)
arr7
```

Output:

56

Code:

```
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([6, 7, 8, 9, 10])
#Starting index in slice is 1 as first even element(2) in array1 is at index 1
even_arr1 = arr1[1:5:2]
print("even for array1",even_arr1)
#Starting index in slice is 0 as first odd element(1) in array1 is at index 0
odd_arr1=arr1[0:5:2]
print("odd for array1",odd_arr1)
#Starting index in slice is 0 as first even element(6) in array2 is at index 0
even_arr2 = arr2[0:5:2]
print("even for array2",even_arr2)
#Starting index in slice is 1 as first odd element(7) in array2 is at index 1
```

```
odd_arr2=arr2[1:5:2]
print("odd for array2",odd_arr2)
```

Output:

```
even for array1 [2 4]
odd for array1 [1 3 5]
even for array2 [ 6 8 10]
odd for array2 [7 9]
```

Lab - Working with 2D Numpy Arrays

Code:

```
import numpy as np
```

Output:

No output

Code:

```
a = [[11, 12, 13], [21, 22, 23], [31, 32, 33]]
a
```

Output:

```
[[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

Code:

```
A = np.array(a)
A
```

Output:

```
array([[11, 12, 13],
       [21, 22, 23],
       [31, 32, 33]])
```

Code:

```
A.ndim
```

Output:

```
2
```

Code:

```
A.shape
```

Output:

```
(3, 3)
```

Code:

```
A.size
```

Output:

```
9
```

Code:

```
A[1, 2]
```

Output:

```
23
```

Code:

```
A[1][2]
```

Output:

```
23
```

Code:

```
A[0][0]
```

Output:

```
11
```

Code:

```
A[0][0:2]
```

Output:

```
array([11, 12])
```

Code:

```
A[0:2, 2]
```

Output:

```
array([13, 23])
```

Code:

```
X = np.array([[1, 0], [0, 1]])
```

```
X
```

Output:

```
array([[1, 0],  
       [0, 1]])
```

Code:

```
Y = np.array([[2, 1], [1, 2]])
```

```
Y
```

Output:

```
array([[2, 1],  
       [1, 2]])
```

Code:

```
Z = X + Y
```

```
Z
```

Output:

```
array([[3, 1],  
       [1, 3]])
```

Code:

```
Y = np.array([[2, 1], [1, 2]])
```

```
Y
```

Output:

```
array([[2, 1],  
       [1, 2]])
```

Code:

```
Z = 2 * Y
```

```
Z
```

Output:

```
array([[4, 2],  
       [2, 4]])
```

Code:

```
Y = np.array([[2, 1], [1, 2]])  
Y
```

Output:

```
array([[2, 1],  
       [1, 2]])
```

Code:

```
X = np.array([[1, 0], [0, 1]])  
X
```

Output:

```
array([[1, 0],  
       [0, 1]])
```

Code:

```
Z = X * Y  
Z
```

Output:

```
array([[2, 0],  
       [0, 2]])
```

Code:

```
A = np.array([[0, 1, 1], [1, 0, 1]])  
A
```

Output:

```
array([[0, 1, 1],  
       [1, 0, 1]])
```

Code:

```
B = np.array([[1, 1], [1, 1], [-1, 1]])
```

B

Output:

```
array([[ 1,  1],
       [ 1,  1],
       [-1,  1]])
```

Code:

```
Z = np.dot(A,B)
```

Z

Output:

```
array([[0, 2],
       [0, 2]])
```

Code:

```
np.sin(Z)
```

Output:

```
array([[0.      , 0.90929743],
       [0.      , 0.90929743]])
```

Code:

```
C = np.array([[1,1],[2,2],[3,3]])
```

C

Output:

```
array([[1, 1],
       [2, 2],
       [3, 3]])
```

Code:

C.T

Output:

```
array([[1, 2, 3],  
       [1, 2, 3]])
```

Code:

```
a = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

```
A = np.array(a)
```

```
A
```

Output:

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

Code:

```
A.size
```

Output:

```
12
```

Code:

```
A[0][0:2]
```

Output:

```
array([1, 2])
```

Code:

```
B = np.array([[0, 1], [1, 0], [1, 1], [-1, 0]])
```

```
X = np.dot(A,B)
```

```
X
```

Output:

```
array([[ 1,  4],  
       [ 5, 12],  
       [ 9, 20]])
```


Code:

```
!pip install Pillow
```

Output:

Collecting Pillow

Downloading pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)

Downloading pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9 MB)

Lab - HTTP and Requests**Code:**

```
import requests
```

Output:

No output

Code:

```
import os  
from PIL import Image  
from IPython.display import IFrame
```

Output:

No output

Code:

```
url='https://www.ibm.com/'  
r=requests.get(url)
```

Output:

No output

Code:

```
r.status_code
```

Output:

200

Code:

```
print(r.request.headers)
```

Output:

```
{'User-Agent': 'python-requests/2.32.3', 'Accept-Encoding': 'gzip, deflate, br, zstd',  
'Accept': '*/.*', 'Connection': 'keep-alive', 'Cookie':  
'_abck=5699A62BF4C2617CF424BEA814D642AB~-  
1~YAAQqYnMF8Tvy7afAQAA5IR99hAJrVoY7ZzM/Gm4C13Qcl8alelx9wrsWnVa81bgjyGh  
Z6fVyDFxRMeBv/ZCPv6n20SUDQ2If5tgFbUGESK0BlptAZjbh9F9qk2S19dam+ClX9jhnjN  
ZZQUZcznbejC/Jmr/jxNzsHqisDhq55SxXREMP2Z9DivrbzXfCZwgtr8E43YJD/e3WdBlRI  
1ZUCP7MzSolqWN4c9PDjfTRqlpquDd9LKBHgKgoRM3YchgQKYKLDCGSJFHlz5Rr14VK1  
qni3pL69AZnaqAxKHaecJOyH56FKyE/tMMeEHllfmcUt2BCkcUsTg5qMPgSw+VmHSwpq  
MWqvlfVeazHA5mYVr6FgAXr6qGuyHal3PEMsgQfAgHtbsg8h5qtizhYVL9J3ZTwTcPRR14  
MdYssLAH+YvWzyxtCXiXm0Lc54F+FFLLQ=~-1~-1~-1~-1~-1;  
bm_sz=278EE7CCCD79D4EEE7ABDCE893C6FA5A~YAAQqYnMF8Xvy7afAQAA5IR99gC  
A2nGoy+/agyZlbhnZsWxYnoX+ngpso7bNwJ4JbSUH9hBEp9RxbLNjV/oNAUzwjfUdy2Tyjdt  
6XHjGgEMQJQmTxQy6T+uojv4ujX4vnnYpTT/v3fOQJ12KxcA+6hQzwbGfTmHuyN/1T5rrMz  
Sc3bBl/p2291J8OyCvfjz/2HvrztD3yx82E7Dtc010Z8siOgzjZcdFL/1fido5vPK2fmyvRKFWg  
0PSsfBZXQBGNyU5vsiCMsRgWUbnpqTNqN9zXTCuXOK39dR22A54P6UHIKWcMJA4Ww  
J1AkwUbmrtEqyYuGRViBWJUu7QPCGrHVkFt9jdAHRbuHl7~4473649~3683636'}
```

Code:

```
print("request body:", r.request.body)
```

Output:

No output

Code:

```
header=r.headers
```

```
print(r.headers)
```

Output:

```
{'Content-Security-Policy': 'upgrade-insecure-requests', 'x-frame-options':  
'SAMEORIGIN', 'Last-Modified': 'Wed, 12 Aug 2026 14:56:16 GMT', 'ETag': '"322fa-  
658daca1612ff-gzip"', 'Accept-Ranges': 'bytes', 'Content-Type': 'text/html; charset=utf-8',  
'X-Content-Type-Options': 'nosniff', 'Cache-Control': 'max-age=114', 'Expires': 'Wed, 12  
Aug 2026 15:02:43 GMT', 'X-Akamai-Transformed': '0 - 0 -', 'Content-Encoding': 'gzip',  
'Date': 'Wed, 12 Aug 2026 15:00:49 GMT', 'Content-Length': '40489', 'Connection': 'keep-  
alive', 'Vary': 'Accept-Encoding', 'Strict-Transport-Security': 'max-age=31536000'}
```

Code:

```
header['Date']
```

Output:

'Wed, 12 Aug 2026 15:00:49 GMT'

Code:

```
header['Content-Type']
```

Output:

```
'text/html; charset=utf-8'
```

Code:

r.encoding

Output:

'utf-8'

Code:

```
r.text[0:100]
```

Output:

```
'\n<!DOCTYPE HTML>\n\n<html lang="en">\n<head>\r\n\r\n\r\n\r\n\r\n\r\n\r\n\r\n\r\n'
```

Code:

```
url='https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0101EN-
SkillsNetwork/IDSNlogo.png'
```

Output:

No output

Code:

```
r=requests.get(url)
```

Output:

No output

Code:

```
print(r.headers)
```

Output:

```
{'Date': 'Wed, 12 Aug 2026 15:07:51 GMT', 'X-Clv-Request-Id': '9fc6b584-c8e2-4b8b-9821-2ea9e66f95b5', 'Server': 'Cleversafe', 'X-Clv-S3-Version': '2.5', 'Accept-Ranges': 'bytes', 'x-amz-request-id': '9fc6b584-c8e2-4b8b-9821-2ea9e66f95b5', 'ETag': '"8bb44578fff8fdcc3d2972be9ece0164"', 'Content-Type': 'image/png', 'Last-Modified': 'Wed, 16 Nov 2022 03:32:41 GMT', 'Content-Length': '78776'}
```

Code:

```
r.headers['Content-Type']
```

Output:

```
'image/png'
```

Code:

```
path=os.path.join(os.getcwd(),'image.png')
```

```
path
```

Output:

```
'/resources/PY0101EN/labs/Module 5/image.png'
```

Code:

```
with open(path,'wb') as f:
```

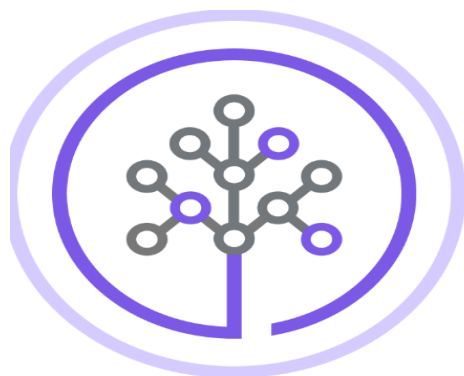
```
    f.write(r.content)
```

Output:

```
No output
```

Code:

```
Image.open(path)
```

Output:

Skills
Network

Code:

```
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0101EN-SkillsNetwork/labs/Module%205/data/Example1.txt'
```

```
path=os.path.join(os.getcwd(),'example1.txt')
```

```
r=requests.get(url)
```

```
with open(path,'wb') as f:
```

```
    f.write(r.content)
```

Output:

No output

Code:

```
url_get='http://httpbin.org/get'
```

Output:

No output

Code:

```
payload={"name":"Joseph","ID":"123"}
```

Output:

No output

Code:

```
r=requests.get(url_get,params=payload)
```

Output:

No output

Code:

```
r.url
```

Output:

[503 Service Temporarily Unavailable](#)

Code:

```
print("request body:", r.request.body)
```

Output:

No output

Code:

```
print(r.status_code)
```

Output:

503

Code:

```
print(r.text)
```

Output:

```
<html>
```

```
<head><title>503 Service Temporarily Unavailable</title></head>
```

```
<body>
```

```
<center><h1>503 Service Temporarily Unavailable</h1></center>
```

```
</body>
```

```
</html>
```

Code:

```
r.headers['Content-Type']
```

Output:

```
'text/html'
```

Code:

```
url_post='http://httpbin.org/post'
```

Output:

No output

Code:

```
r_post=requests.post(url_post,data=payload)
```

Output:

No output

Code:

```
print("POST request URL:",r_post.url )  
print("GET request URL:",r.url)
```

Output:

POST request URL: <http://httpbin.org/post>

GET request URL: <http://httpbin.org/get?name=Joseph&ID=123>

Code:

```
print("POST request URL:", r_post.url)
```

Output:

POST request URL: <http://httpbin.org/post>

Lab - API**Code:**

```
!pip install randomuser
```

```
!pip install pandas
```

Output:

Requirement already satisfied: randomuser in /opt/conda/lib/python3.12/site-packages (1.6)

Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-packages (3.0.5)

Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-packages (from pandas) (2.5.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Code:

```
from randomuser import RandomUser  
  
import pandas as pd
```

Output:

No output

Code:

```
r = RandomUser()
```

Output:

No output

Code:

```
some_list = r.generate_users(10)
```

Output:

No output

Code:

```
some_list
```

Output:

```
[<randomuser.RandomUser at 0x7131f9aaa2d0>,  
<randomuser.RandomUser at 0x7131f9aaa300>,  
<randomuser.RandomUser at 0x7131f9aaa330>,  
<randomuser.RandomUser at 0x7131f9aaa360>,  
<randomuser.RandomUser at 0x7131f9aaa390>,  
<randomuser.RandomUser at 0x7131f9aaa3c0>,  
<randomuser.RandomUser at 0x7131f9aaa3f0>,  
<randomuser.RandomUser at 0x7131f9aaa420>,  
<randomuser.RandomUser at 0x7131f9aaa450>,  
<randomuser.RandomUser at 0x7131f9aaa480>]
```

Code:

```
name = r.get_full_name()
```

Output:

No output

Code:

for user in some_list:

```
    print (user.get_full_name()," ",user.get_email())
```

Output:

Maja Schwanke maja.schwanke@example.com

Melisa Leroux melisa.leroux@example.com

Ryder Chu ryder.chu@example.com

Quaresma Ferreira quaresma.ferreira@example.com

Amelia Taylor amelia.taylor@example.com

Lena Denis lena.denis@example.com

Zoe Berry zoe.berry@example.com

Vernon Reed vernon.reed@example.com

Bently Gagnon bently.gagnon@example.com

Mario Volkert mario.volkert@example.com

Code:

for user in some_list:

```
    print (user.get_picture())
```

Output:

<https://randomuser.me/api/portraits/women/92.jpg>

<https://randomuser.me/api/portraits/women/95.jpg>

<https://randomuser.me/api/portraits/men/17.jpg>

<https://randomuser.me/api/portraits/men/47.jpg>

<https://randomuser.me/api/portraits/women/9.jpg>

<https://randomuser.me/api/portraits/women/93.jpg>

<https://randomuser.me/api/portraits/women/76.jpg>

<https://randomuser.me/api/portraits/men/77.jpg>

<https://randomuser.me/api/portraits/men/70.jpg>

<https://randomuser.me/api/portraits/men/84.jpg>

Code:

```
def get_users():

    users =[]

    for user in RandomUser.generate_users(10):

users.append({"Name":user.get_full_name(),"Gender":user.get_gender(),"City":user.get_
city(),"State":user.get_state(),"Email":user.get_email(),
"DOB":user.get_dob(),"Picture":user.get_picture()})

    return pd.DataFrame(users)
```

Output:

	Name	Gender	City	State	Email	DOB	Picture
0	Yasemin Baturalp	female	Batman	Bitlis	yasemin.baturalp@example.com	1974-09-23T13:36:35.137Z	https://randomuser.me/api/portraits/women/55.jpg
1	Julia Olson	female	Los Angeles	Rhode Island	julia.olson@example.com	1964-10-16T13:15:11.157Z	https://randomuser.me/api/portraits/women/17.jpg
2	Victoria Martinez	female	Lyon	Loire-Atlantique	victoria.martinez@example.com	1991-11-09T00:13:53.857Z	https://randomuser.me/api/portraits/women/31.jpg
3	Lloyd Chapman	male	Athens	Pennsylvania	lloyd.chapman@example.com	1946-03-07T09:07:39.919Z	https://randomuser.me/api/portraits/men/90.jpg
4	Pompeu Pereira	male	Sete Lagoas	Piauí	pompeu.pereira@example.com	1996-06-22T21:50:43.733Z	https://randomuser.me/api/portraits/men/94.jpg
5	Emre Van Veldhoven	male	Blesdijke	Groningen	emre.vanveldhoven@example.com	1994-06-28T01:23:11.170Z	https://randomuser.me/api/portraits/men/71.jpg
6	Esperanza Soler	female	Ferrol	Aragón	esperanza.soler@example.com	1976-10-30T21:48:30.768Z	https://randomuser.me/api/portraits/women/55.jpg
7	Emil Johansen	male	St.Merløse	Midtjylland	emil.johansen@example.com	1946-09-21T13:51:12.714Z	https://randomuser.me/api/portraits/men/30.jpg
8	Annemaria Henssen	female	Boxmeer	Overijssel	annemaria.henssen@example.com	1975-06-12T12:54:11.130Z	https://randomuser.me/api/portraits/women/54.jpg
9	Thi Barbier	female	Riedholz	Uri	thi.barbier@example.com	1988-06-25T09:57:43.289Z	https://randomuser.me/api/portraits/women/65.jpg

Code:

```
df1 = pd.DataFrame(get_users())
```

Output:

No output

Code:

```
import requests
```

```
import json
```

Output:

No output

Code:

```
data=requests.get("https://web.archive.org/web/20240929211114/https://fruityvice.com/api/fruit/all")
```

Output:

No output

Code:

```
results = json.loads(data.text)
```

Output:

No output

Code:

```
pd.DataFrame(results)
```

Output:

	name	id	family	order	genus	nutritions
0	Persimmon	52	Ebenaceae	Rosales	Diospyros	{'calories': 81, 'fat': 0.0, 'sugar': 18.0, 'c...
1	Strawberry	3	Rosaceae	Rosales	Fragaria	{'calories': 29, 'fat': 0.4, 'sugar': 5.4, 'ca...
2	Banana	1	Musaceae	Zingiberales	Musa	{'calories': 96, 'fat': 0.2, 'sugar': 17.2, 'c...
3	Tomato	5	Solanaceae	Solanales	Solanum	{'calories': 74, 'fat': 0.2, 'sugar': 2.6, 'ca...
4	Pear	4	Rosaceae	Rosales	Pyrus	{'calories': 57, 'fat': 0.1, 'sugar': 10.0, 'c...
5	Durian	60	Malvaceae	Malvales	Durio	{'calories': 147, 'fat': 5.3, 'sugar': 6.75, '...
6	Blackberry	64	Rosaceae	Rosales	Rubus	{'calories': 40, 'fat': 0.4, 'sugar': 4.5, 'ca...
7	Lingonberry	65	Ericaceae	Ericales	Vaccinium	{'calories': 50, 'fat': 0.34, 'sugar': 5.74, '...
8	Kiwi	66	Actinidiaceae	Struthioniformes	Apteryx	{'calories': 61, 'fat': 0.5, 'sugar': 9.0, 'ca...
9	Lychee	67	Sapindaceae	Sapindales	Litchi	{'calories': 66, 'fat': 0.44, 'sugar': 15.0, '...
10	Pineapple	10	Bromeliaceae	Poales	Ananas	{'calories': 50, 'fat': 0.12, 'sugar': 9.85, '...
11	Fig	68	Moraceae	Rosales	Ficus	{'calories': 74, 'fat': 0.3, 'sugar': 16.0, 'c...
12	Gooseberry	69	Grossulariaceae	Saxifragales	Ribes	{'calories': 44, 'fat': 0.6, 'sugar': 0.0, 'ca...
13	Passionfruit	70	Passifloraceae	Malpighiales	Passiflora	{'calories': 97, 'fat': 0.7, 'sugar': 11.2, 'c...
14	Plum	71	Rosaceae	Rosales	Prunus	{'calories': 46, 'fat': 0.28, 'sugar': 9.92, '...
15	Orange	2	Rutaceae	Sapindales	Citrus	{'calories': 43, 'fat': 0.2, 'sugar': 8.2, 'ca...
16	GreenApple	72	Rosaceae	Rosales	Malus	{'calories': 21, 'fat': 0.1, 'sugar': 6.4, 'ca...
17	Raspberry	23	Rosaceae	Rosales	Rubus	{'calories': 53, 'fat': 0.7, 'sugar': 4.4, 'ca...
18	Watermelon	25	Cucurbitaceae	Cucurbitales	Citrullus	{'calories': 30, 'fat': 0.2, 'sugar': 6.0, 'ca...
19	Lemon	26	Rutaceae	Sapindales	Citrus	{'calories': 29, 'fat': 0.3, 'sugar': 2.5, 'ca...
20	Mango	27	Anacardiaceae	Sapindales	Mangifera	{'calories': 60, 'fat': 0.38, 'sugar': 13.7, '...
21	Blueberry	33	Rosaceae	Rosales	Fragaria	{'calories': 29, 'fat': 0.4, 'sugar': 5.4, 'ca...
22	Apple	6	Rosaceae	Rosales	Malus	{'calories': 52, 'fat': 0.4, 'sugar': 10.3, 'c...
23	Guava	37	Myrtaceae	Myrtales	Psidium	{'calories': 68, 'fat': 1.0, 'sugar': 9.0, 'ca...
24	Apricot	35	Rosaceae	Rosales	Prunus	{'calories': 15, 'fat': 0.1, 'sugar': 3.2, 'ca...
25	Melon	41	Cucurbitaceae	Cucurbitaceae	Cucumis	{'calories': 34, 'fat': 0.0, 'sugar': 8.0, 'ca...
26	Tangerine	77	Rutaceae	Sapindales	Citrus	{'calories': 45, 'fat': 0.4, 'sugar': 9.1, 'ca...
27	Pitahaya	78	Cactaceae	Caryophyllales	Cactaceae	{'calories': 36, 'fat': 0.4, 'sugar': 3.0, 'ca...
28	Lime	44	Rutaceae	Sapindales	Citrus	{'calories': 25, 'fat': 0.1, 'sugar': 1.7, 'ca...
29	Pomegranate	79	Lythraceae	Myrtales	Punica	{'calories': 83, 'fat': 1.2, 'sugar': 13.7, 'c...
30	Dragonfruit	80	Cactaceae	Caryophyllales	Selenicereus	{'calories': 60, 'fat': 1.5, 'sugar': 8.0, 'ca...
31	Grape	81	Vitaceae	Vitales	Vitis	{'calories': 69, 'fat': 0.16, 'sugar': 16.0, '...
32	Morus	82	Moraceae	Rosales	Morus	{'calories': 43, 'fat': 0.39, 'sugar': 8.1, 'c...
33	Feijoa	76	Myrtaceae	Myrtoideae	Sellowiana	{'calories': 44, 'fat': 0.4, 'sugar': 3.0, 'ca...
34	Avocado	84	Lauraceae	Laurales	Persea	{'calories': 160, 'fat': 14.66, 'sugar': 0.66,...
35	Kiwifruit	85	Actinidiaceae	Ericales	Actinidia	{'calories': 61, 'fat': 0.5, 'sugar': 8.9, 'ca...
36	Cranberry	87	Ericaceae	Ericales	Vaccinium	{'calories': 46, 'fat': 0.1, 'sugar': 4.0, 'ca...
37	Cherry	9	Rosaceae	Rosales	Prunus	{'calories': 50, 'fat': 0.3, 'sugar': 8.0, 'ca...
38	Peach	86	Rosaceae	Rosales	Prunus	{'calories': 39, 'fat': 0.25, 'sugar': 8.4, 'c...
39	Jackfruit	94	Moraceae	Rosales	Artocarpus	{'calories': 95, 'fat': 0.0, 'sugar': 19.1, 'c...
40	Horned Melon	95	Cucurbitaceae	Cucurbitales	Cucumis	{'calories': 44, 'fat': 1.26, 'sugar': 0.5, 'c...

Code:

```
df2 = pd.json_normalize(results)
```

Output:

No output

Code:

```
df2
```

Output:

	name	id	family	order	genus	nutrition.calories	nutrition.fat	nutrition.sugar	nutrition.carbohydrates	nutrition.protein
0	Persimmon	52	Ebenaceae	Rosales	Diospyros	81	0.00	18.00	18.00	0.00
1	Strawberry	3	Rosaceae	Rosales	Fragaria	29	0.40	5.40	5.50	0.80
2	Banana	1	Musaceae	Zingiberales	Musa	96	0.20	17.20	22.00	1.00
3	Tomato	5	Solanaceae	Solanales	Solanum	74	0.20	2.60	3.90	0.90
4	Pear	4	Rosaceae	Rosales	Pyrus	57	0.10	10.00	15.00	0.40
5	Durian	60	Malicaceae	Malvales	Durio	147	5.30	6.75	27.10	1.50
6	Blackberry	61	Rosaceae	Rosales	Rubus	40	0.40	4.50	9.00	1.20
7	Lingonberry	65	Ericaceae	Ericales	Vaccinium	50	0.34	5.74	11.20	0.75
8	Kivi	66	Actinidiaceae	Smurthioniformes	Actinidia	61	0.50	9.00	15.00	1.10
9	Lychee	67	Sapindaceae	Sapindales	Litchi	66	0.44	15.00	17.00	0.80
10	Pineapple	10	Bromeliaceae	Posales	Ananas	50	0.12	9.85	13.12	0.54
11	Fig	68	Moraceae	Rosales	Ficus	74	0.30	16.00	19.00	0.60
12	Gooseberry	69	Grossulariaceae	Sorothogales	Ribes	44	0.60	0.00	10.00	0.90
13	Passionfruit	70	Passifloraceae	Mulpighiales	Passiflora	67	0.70	11.20	22.40	2.20
14	Plum	71	Rosaceae	Rosales	Prunus	46	0.26	9.92	11.40	0.70
15	Orange	2	Rutaceae	Sapindales	Citrus	43	0.20	8.20	8.30	1.00
16	GreenApple	72	Rosaceae	Rosales	Malus	21	0.10	6.40	3.10	0.40
17	Raspberry	23	Rosaceae	Rosales	Rubus	53	0.70	4.40	12.00	1.20
18	Watermelon	25	Cucurbitaceae	Cucurbitales	Citrullus	30	0.20	6.00	8.00	0.60
19	Lemon	26	Rutaceae	Sapindales	Citrus	29	0.30	2.50	9.00	1.10
20	Mango	27	Anacardiaceae	Sapindales	Mangifera	60	0.38	13.70	15.00	0.82
21	Blueberry	33	Rosaceae	Rosales	Fragaria	29	0.40	5.40	5.50	0.80
22	Apple	6	Rosaceae	Rosales	Malus	52	0.40	10.30	11.40	0.30
23	Guaia	37	Myrtaceae	Myrtales	Podium	68	1.00	9.00	14.00	2.60
24	Apricot	35	Rosaceae	Rosales	Prunus	15	0.10	3.20	3.90	0.50
25	Melon	41	Cucurbitaceae	Cucurbitaceae	Cucumis	34	0.00	8.00	8.00	0.00
26	Tangerine	77	Rutaceae	Sapindales	Citrus	45	0.40	9.10	8.30	0.00
27	Pitahaya	78	Cactaceae	Caryophyllales	Cactaceae	36	0.40	3.00	7.00	1.00
28	Lime	44	Rutaceae	Sapindales	Citrus	25	0.10	1.70	8.40	0.30
29	Pomegranate	79	Lythaceae	Myrtales	Punica	63	1.20	13.70	16.70	1.70
30	Dragonfruit	80	Cactaceae	Caryophyllales	Selenicereus	60	1.50	8.00	9.00	9.00
31	Grape	81	Vitaceae	Vitales	Vitis	69	0.16	16.00	18.10	0.72
32	Monst	82	Moraceae	Rosales	Monst	43	0.26	8.10	9.80	1.44
33	Felipa	76	Myrtaceae	Myrtales	Sellowiana	44	0.40	3.00	8.00	0.60
34	Avocado	84	Lauraceae	Laurales	Persea	160	14.66	0.66	8.53	2.00
35	Kiwifruit	85	Actinidiaceae	Ericales	Actinidia	61	0.50	8.90	14.60	1.14
36	Cranberry	87	Ericaceae	Ericales	Vaccinium	46	0.10	4.00	12.20	0.40
37	Cherry	9	Rosaceae	Rosales	Prunus	50	0.30	8.00	12.00	1.00
38	Peach	86	Rosaceae	Rosales	Prunus	39	0.25	8.40	9.50	0.90
39	Jackfruit	94	Moraceae	Rosales	Artocarpus	95	0.00	19.10	23.20	1.72
40	Honeyd Melon	95	Cucurbitaceae	Cucurbitales	Cucumis	44	1.26	0.50	7.56	1.78
41	Hazelnut	96	Betulaceae	Fagales	Corylus	628	61.80	4.30	17.00	15.00
42	Pomeelo	98	Rutaceae	Sapindales	Citrus	37	0.00	0.50	9.67	0.82
43	Mangosteen	99	Clusiaceae	Mulpighiales	Garcinia	73	0.58	16.11	17.91	0.41
44	Pumpkin	100	Cucurbitaceae	Cucurbitales	Cucurbita	25	0.30	3.30	4.60	1.10
45	Japanese Persimmon	101	Ebenaceae	Ericales	Diospyros	70	0.20	13.00	19.00	0.60
46	Papaya	42	Caricaceae	Bresicales	Carica	39	0.30	4.40	5.80	0.50
47	Annona	103	Annonaceae	Rosales	Annonas	92	0.29	3.40	19.10	1.50
48	Ceylon Gooseberry	104	Salicaceae	Mulpighiales	Diospylos	47	0.30	8.10	9.60	1.20

Code:

```
cherry = df2.loc[df2["name"] == 'Cherry']  
(cherry.iloc[0]['family']) , (cherry.iloc[0]['genus'])
```

Output:

('Rosaceae', 'Prunus')

Code:

```
cal_banana = df2.loc[df2["name"] == 'Banana']  
cal_banana.iloc[0]['nutritions.calories']
```

Output:

np.int64(96)

Code:

```
data2 = requests.get("https://official-joke-api.appspot.com/jokes/ten")
```

Output:

No output

Code:

```
results2 = json.loads(data2.text)
```

Output:

No output

Code:

```
df3 = pd.DataFrame(results2)  
df3.drop(columns=["type","id"],inplace=True)  
df3
```

Output:

	setup	punchline
0	What's the difference between an African eleph...	About 5000 miles.
1	Why did the scarecrow win an award?	Because he was outstanding in his field.
2	How many hipsters does it take to change a lig...	Oh, it's a really obscure number. You've proba...
3	What do you call a thieving alligator?	A crookodile!
4	What do elves post on Social Media?	Elf-ies.
5	What do you call a suspicious looking laptop?	Asus
6	What did the fish say when it swam into a wall?	Damn!
7	Why do ducks make great detectives?	They always quack the case.
8	Did you hear that the police have a warrant ou...	It reads "Small medium at large."
9	Why did the cookie go to the doctor?	Because it was feeling crumbly.

Code:

```
import numpy as np  
A=np.array([[11,12],[21,22],[31,32]])
```

Output:

No output

Code:

```
type(A)
```

Output:

numpy.ndarray

Code:

```
A.shape
```

Output:

(3, 2)

Code:

```
A.dtype
```

Output:

dtype('int64')

Code:

```
A[1]
```

Output:

array([21, 22])

Code:

```
A=np.array([[11,12],[21,22]])  
B=np.array([[1, 0],[0,1]])
```

Output:

No output

Code:

```
C = A * B
```


C

Output:

```
array([[11, 0],  
       [ 0, 22]])
```

Code:

```
Z = np.dot(A,B)
```

Z

Output:

```
array([[11, 12],  
       [21, 22]])
```